Reviewing What You Never Wrote: Memory-Model Literacy and the Human Review of AI-Generated Code at the Python/C Boundary

Jessica Dye
Niagara University
Niagara University, NY, USA

Abstract—As AI becomes increasingly capable of writing and debugging software, what happens when both the developer and the AI are working with abstractions built on lower-level systems that the developer doesn't understand? We take a pro-AI position: the question is not whether to use these tools, but what human understanding is needed to oversee them. The Python libraries that AI and scientific computing rely on are built on C, and that C is increasingly written by AI. We study the seam where the question is sharpest: code that crosses the Python/C memory-management boundary. There, reference counting is CPython's garbage collector, and the collector's bookkeeping becomes the programmer's job. We propose a three-level construct of memory-model literacy (managed, manual, and boundary-contract) and a two-part study. First, a corpus study asks which defects in LLM-generated boundary code survive sanitizers, static analysis, and LLM review. Second, a student study asks whether literacy at each level predicts catching those defects better than years of experience does. A motivating example shows a dangling-reference bug that two standard static analyzers do not flag.

I. Problem

The abstraction runs on C. Students increasingly learn computing through Python, but the stack they use is largely native code. By our count, 55.7% of NumPy's source lines are C/C++ and 47.4% of CPython's (lines classified by file extension, vendored code excluded). Yang and Cai report 38% for NumPy under their own counting rule [1]; under either rule the native core is substantial. Memory-safety defects remain the most prevalent class of disclosed vulnerability in memory-unsafe code [2]. Vulnerability proneness in multilingual code tracks the interfacing mechanism, not just the language [3].

The seam is where it breaks. In a study of 216 native-code bugs in Python applications, reference counting was involved in 41.7%, and foreign-object handling and cross-language exception handling in 31.5% each [1]. These are not ordinary C mistakes. They are violations of a contract between C code and the Python runtime: which references are new, borrowed, or stolen; when an exception is set; which allocator owns which memory [4].

That code is increasingly written by AI, and reviewed by humans who may not know the contract. LLM-generated C has well-documented memory-safety defects [5], though results on human-plus-AI code are mixed [6]. Developers using AI assistants can write less secure code while feeling more confident [7]. Human review is the stated safeguard, but it is weak: experience did not predict security-review effectiveness [8]. In the closest prior work, 100 developers could not reliably distinguish secure from insecure AI-generated C, and more experienced developers submitted more vulnerabilities [9]. Meanwhile, the junior roles where developers traditionally learned this contract under review are contracting. Brynjolfsson et al. report a 16% relative employment decline for 22–25-year-olds in AI-exposed occupations [10]. The causes are contested, and we claim no more than that this pathway is under pressure.

Gap. Prior human studies measured experience and found it wanting. Prior education work linked students' knowledge of memory to writing secure C [11]. We found no prior evaluation of the security of LLM-generated Python/C API code; a 2026 study of LLM-generated bindings notes that no benchmark for binding generation exists [12]. Nobody has asked what a reviewer must understand to catch what AI leaves at the boundary.

Guiding question. As AI becomes increasingly capable of writing and debugging software, what happens when both the developer and the AI are working with abstractions built on lower-level systems that the developer doesn't understand? We answer it in three parts. SQ1: How much of the Python AI and scientific stack is built on native code? SQ2: Which defects in AI-generated Python/C code do automated tools, AI reviewers included, miss? SQ3: Does a developer's understanding of the memory model predict whether they catch what the tools miss? SQ1 is answered above by our repository counts; SQ2 and SQ3 are the studies in Section IV.

II. Construct: Three Levels of Memory-Model Literacy

Table I defines the construct. It targets a common misconception: that a garbage-collected language makes memory someone else's problem. Garbage collection does not eliminate memory-related vulnerabilities; it relocates them. Its guarantees hold only for code that stays inside the managed runtime, and native extensions void them [13]. Our hypothesis: the defects that survive automated review are dominated by L3, and L3 depends on L2 and cannot be inferred from L1 alone. If so, students

TABLE I
Levels of memory-model literacy

Level	The reader understands	Example question
L1 Managed	References, aliasing, mutation, and lifetime in a garbage-collected language	After b = a; b.append(1), what is a?
L2 Manual	Allocation and release, ownership, dangling and null pointers, matching free to allocator	Who frees this buffer, and with which function?
L3 Boundary	The collector's guarantees and the obligations C code takes on to keep them: new, borrowed, and stolen references; exception state; allocator families	This call returns a borrowed reference. What happens if a callback runs before it is used?

who only ever see a managed language are missing exactly the knowledge needed to review the code their tools are built from.

III. Motivating Example

The following function, written by the author to illustrate a defect pattern from the literature [4], packs the first element of a list into a new tuple:

```
PyObject *t = PyTuple_New(1);
PyTuple_SET_ITEM(t, 0, PyList_GET_ITEM(lst, 0));
return t;   /* missing Py_INCREF on the item */
```

PyList_GET_ITEM returns a borrowed reference; PyTuple_SET_ITEM steals one. When the tuple is freed, the item's count drops below what the list expects, leaving the list holding a dangling pointer. The code compiles cleanly and looks idiomatic. Neither cppcheck 2.22 (–enable=all) nor clang-tidy 23.1 (all checks) reported the defect; clang-tidy emitted only style warnings. On a release build of CPython 3.13 the program crashed with a bus error and no diagnostic. Only a debug build of CPython (negative-refcount assertion) and AddressSanitizer, given the right input, exposed it. Recognizing the bug from the source requires L3: knowing which calls borrow and which steal.

This is one hand-written example, not evidence about AI. Whether LLMs produce such defects, how often, and whether a full automated tier misses them is the empirical question.

IV. Research Plan

Study 1: Corpus (SQ2). We ask which defects survive the combined automated tier, and, for each survivor, the lowest literacy level at which it is recognizable. Realistic developer requests, phrased like Stack Overflow questions with no security hints, that require C-API extensions, ctypes/cffi wrappers, or exception propagation from C. We have written ten pilot prompts. Each prompt goes to two current models (Claude and GPT), three samples each, logging model version and date; the corpus is frozen and released. The automated tier combines AddressSanitizer/UBSan and a debug CPython under edge-case inputs, cppcheck and clang-tidy, and an LLM reviewer from a vendor other than the generating model. Ground truth is a manual audit with a codebook drawn from Yang and Cai's taxonomy [1] and Hu and Zhang's bug patterns [4]. Because static analyzers agree with human labels on only about 61–65% of samples [14], labels are triangulated: a defect counts if a sanitizer or assertion reproduces it, or if two coders agree. The residue is every defect missed by all tools; each is coded L1, L2, or L3.

Study 2: Students (SQ3, IRB pending). We ask whether L1/L2/L3 scores predict detection better than years of experience does, and how well calibrated students' confidence is. About 15–30 undergraduates, from CS1 through senior year, take a short literacy test, then review real defective snippets from the residue plus clean controls. Following prior method work, the task is reading rather than writing [15], every participant is told to look for security problems [16], and no AI review is shown. Participants rate their confidence in each verdict. We report detection by literacy level, how strongly literacy scores and years of experience each track detection, and calibration, as a pilot with effect sizes. A task closer to practice, where participants use an AI assistant to speed up a slow Python routine in C and we test what they ship [6], is future work.

Defeat conditions. The hypothesis fails if (1) automation catches nearly everything, leaving little residue; (2) L1 predicts detection as well as L3; (3) the LLM reviewer alone matches the full tier, reducing the human role to verifying the verifier; or (4) no predictor works, replicating [8], [9] at the boundary. Each is still a reportable result for curriculum design.

Difference from closest prior work. Khalid et al. [9] study plain C and use experience as the predictor. We target the cross-language boundary, characterize what automation leaves behind, and test a specific, teachable construct instead of experience.

V. Questions for the Workshop

(1) Is a three-level construct the right grain, or should L3 be split, for example into reference ownership and exception state? (2) How can a literacy test avoid simply rewarding students who have taken a systems course? (3) Which automated tools would reviewers consider the minimum credible tier?

References

[1] H. Yang and H. Cai, "A Comprehensive Study of Native Code Bugs in Python Applications," 2026, arXiv:2608.29851.

[2] CISA, NSA, FBI, et al., "The Case for Memory Safe Roadmaps," Joint guidance, Dec. 2023.

[3] W. Li, L. Li, and H. Cai, "On the Vulnerability Proneness of Multilingual Code," in Proc. ESEC/FSE, 2022.

[4] M. Hu and Y. Zhang, "The Python/C API: Evolution, Usage Statistics, and Bug Patterns," J. Softw. Evol. Process, vol. 35, no. 2, 2023, extends SANER 2020.

[5] H. Pearce et al., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in IEEE Symposium on Security and Privacy (S&P), 2022.
[6] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants," in Proc. USENIX Security, 2023.
[7] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in Proc. ACM CCS, 2023.
[8] A. Edmundson et al., "An Empirical Study on the Effectiveness of Security Code Review," in Proc. ESSoS, 2013.
[9] H. Khalid, R. E. Thompson, A. Sabater, P. Mussiba, K. R. Fulton, and D. Votipka, "(Don't) Trust, but (Don't) Verify: Developers' Attention to Security in AI-Generated Code," in IEEE Symposium on Security and Privacy (S&P), 2027, to appear. arXiv:2609.21020.
[10] E. Brynjolfsson, B. Chandar, and R. Chen, "Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence," Stanford Digital Economy Lab, 2025.
[11] M. Almansoori, J. Lam, E. Fang, A. G. Soosai Raj, and R. Chatterjee, "Towards Finding the Missing Pieces to Teach Secure Programming Skills to Students," in Proc. ACM SIGCSE, 2023.
[12] "Python Bindings for a Large C++ Robotics Library: The Case of OMPL," 2026, arXiv:2603.04668.
[13] C.-A. Staicu, S. Rahaman, Á. Kiss, and M. Backes, "Bilingual Problems: Studying the Security Risks Incurred by Native Extensions in Scripting Languages," in Proc. USENIX Security, 2023.
[14] E. Firouzi and M. Ghafari, "Persistent Human Feedback, LLMs, and Static Analyzers for Secure Code Generation and Vulnerability Detection," 2026, arXiv:2602.05868.
[15] K. R. Fulton, J. Lewis, N. Malkin, and M. L. Mazurek, "Write, Read, or Fix? Exploring Alternative Methods for Secure Development Studies," in Proc. SOUPS, 2024.
[16] L. Braz, C. Aeberhard, G. Çalikli, and A. Bacchelli, "Less is More: Supporting Developers in Vulnerability Detection during Code Review," in Proc. ICSE, 2022.